COMPTE RENDU

ArtiShop

TP1 / Git

Synchronisation et branches avec Git

ÉQUIPE

Zouhair Fouiguira
Yassine Belhamadia
Zainab Achak
Wiam Bzioui

DÉPÔT DE L'ÉQUIPE

https://github.com/ZOUHAIRFGRA/fsr_ArtiShop

CONTENU DU LIVRABLE

Réponses aux questions et réflexions du TP.
Vérifications techniques et références des commits.
Graphe final intégral du dépôt.

Exercice 1 – Remotes

Q1.1 – Où les remotes sont-ils enregistrés ?

Dans `.git/config`, sous les sections `[remote "origin"]` et `[remote "upstream"]`. Elles contiennent notamment l'URL et les règles de récupération des références.

Q1.2 – Pourquoi le push upstream est-il refusé ?

Le fork donne des droits sur le dépôt de l'équipe, pas sur celui de l'enseignante. Sans autorisation d'écriture sur upstream, le serveur refuse le push. Les contributions sont publiées sur origin.

Réflexion. origin désigne le fork, upstream le dépôt de l'enseignante. Un remote est un nom associé à une URL ; ajouter `backup` avec la même URL ne crée pas une copie supplémentaire du dépôt. Ce remote temporaire peut ensuite être supprimé avec `git remote remove backup`.

Exercice 2 – Clones

Q2.1 – Quel remote est créé automatiquement ?

`git clone` crée origin, associé à l'URL clonée. upstream n'apparaît pas automatiquement : chaque clone doit le configurer séparément s'il souhaite suivre le dépôt de l'enseignante.

Exercice 3 – Fetch et pull

Q3.1 – À quelles opérations équivaut git pull ?

Avec une stratégie de fusion : `git fetch`, puis `git merge` de la branche suivie. Avec `-rebase`, la seconde étape est un rebase.

Q3.2 – Pourquoi fetch est-il qualifié de sûr ?

fetch télécharge les objets et actualise les références distantes sans fusionner dans la branche courante ni modifier les fichiers de travail. On peut examiner les changements avant de les intégrer.

Observations après fetch

Après un fetch, `git status` peut annoncer une branche en retard. Les commits reçus mais non intégrés s'affichent avec :

```
git log main..origin/main --oneline
```

Le README local reste inchangé. La recherche de la section Recette ne la trouve donc pas si elle n'était pas déjà présente. Un merge ou un pull intègre ensuite ces commits.

Exercice 4 – Branches et HEAD détachée

`git branch` affiche les branches locales. `git branch -a` affiche aussi les références de suivi distant.

- `git switch origin/feature/livraison` attend une branche locale et refuse normalement cette référence distante : `a branch is expected, got remote branch`.
- `git checkout origin/feature/livraison` place HEAD directement sur un commit : HEAD est détachée.
- `git switch feature/livraison` sélectionne la branche locale ou peut la créer par suivi d'une branche distante non ambiguë.

Risque. Des commits faits en HEAD détachée ne sont pas attachés à une branche. Après un changement de position, ils peuvent devenir difficiles à retrouver puis être collectés. Créer une branche avant de quitter cet état les conserve.

Exercice 5 – Fusion fast-forward

Q5.1 – Pourquoi aucun commit de fusion n'est-il nécessaire ?

Si la branche cible est ancêtre de la branche intégrée, Git avance son pointeur jusqu'au commit descendant. Il n'y a pas deux histoires divergentes à combiner : c'est un fast-forward. C'est la condition attendue aux étapes 2 et 6 de cet exercice.

Exercice 6 – Fusion à trois voies

Q6.1 – Quels sont les trois commits utilisés ?

Pour la fusion livraison corrigée `09d2a94` :

- base commune : `c0e265d` ;
- ancien sommet de main : `a09b892` ;
- sommet livraison : `080a49c`.

Vérification après suppression de la branche livraison :

```
git merge-base 09d2a94^1 09d2a94^2
git show --no-patch --format="%H %P" 09d2a94
```

Avant la fusion, `git merge-base main feature/livraison` donnait la base commune. Après fusion, cette commande donnait le sommet livraison, devenu ancêtre de main.

Q6.2 – Pourquoi deux modifications du même fichier peuvent-elles fusionner ?

Git compare les modifications à leur base commune. Des changements dans des zones distinctes du fichier peuvent être combinés automatiquement : l'ajout de livraison après `/health` et les changements du panier pour les promos ne sont pas nécessairement concurrents.

Dans ce dépôt, la fusion livraison a néanmoins nécessité une résolution manuelle. Le résultat corrigé contient une seule route livraison et une seule déclaration correspondante ; il ne faut pas décrire cette fusion réelle comme entièrement automatique.

Exercice 7 – Conflit RAMADAN20

Les branches modifient la même ligne de `src/data/promos.js` à partir d'une base commune : 25 % côté marketing, 20 % côté PO. Git ne peut pas décider quelle valeur fonctionnelle garder et signale un conflit.

`git status` indique le fichier non fusionné. Les marqueurs `<<<<<<<`, `=======` et `>>>>>>>` délimitent les versions. La résolution conserve 20 %, supprime les marqueurs, puis se termine par les tests, `git add` et `git commit`. La fusion résultante est `8edf2d4`.

Q7.1 – Pourquoi un fast-forward ne produit-il jamais de conflit de fusion ?

Il déplace un pointeur vers un descendant existant, sans combiner deux versions divergentes. Des modifications locales non commitées peuvent empêcher l'opération, mais il ne s'agit pas d'un conflit entre deux historiques.

Annulation. `git merge -abort` annule une fusion en cours. Il faut conserver un état de travail propre avant la fusion pour faciliter le retour à l'état initial.

Exercice 8 – Suppression des branches

`git branch -d` supprime une branche locale intégrée ; `git push origin -delete nom` supprime la branche sur le fork. Cela ne supprime ni les commits encore accessibles depuis main, ni la branche homonyme chez upstream.

Les références de suivi distant des autres clones peuvent subsister jusqu'à `git fetch -prune origin`. Les branches `feature/promo` et `feature/livraison` existent toujours chez upstream, comme le confirme `git ls-remote -heads upstream`. La branche `feature/avis` correspond au bonus ultérieur.

Exercice 9 – Pull avec rebase

Q9.1 – Qu'aurait donné git pull –no-rebase ?

Avec deux historiques divergents, il aurait effectué un fetch puis une fusion, créant un commit de merge. Le rebase rejoue le commit local après le commit distant et conserve une suite linéaire pour ces changements.

Q9.2 – Quelle est la règle d'or du rebase ?

Ne pas rebaser sans coordination des commits publiés sur lesquels d'autres personnes ont fondé leur travail. Rejouer un commit local non publié évite de réécrire l'histoire utilisée par les autres.

Exercice 9 – Observation du refus

Le refus de push non-fast-forward et la séquence pull/rebase/push ont été reproduits avec un dépôt distant local. Les commits résultants sont publiés sur le fork : correction README `52fe0a0`, puis couleur `450465b`. La couleur principale est `#1f6f78`.

Le serveur refuse le premier push car il contient un commit absent du clone. Après récupération, le rebase place la correction distante avant le commit local ; le push devient alors un fast-forward.

Exercice 10 – Synchronisation upstream

- `git log main..upstream/main -oneline` : commits d'upstream/main absents de main.
- `git log upstream/main..main -oneline` : commits de main absents d'upstream/main.

Un fetch réussi doit précéder l'interprétation de ces listes. La mise à jour `c923300` a été intégrée par la fusion `ea385df`. Après intégration, la première liste est vide.

Sync fork. L'action Update branch de GitHub intègre les changements du dépôt d'origine dans le fork. L'équivalent usuel est : fetch upstream, sélection de main, merge upstream/main, puis push origin main. Une option qui abandonne les commits propres au fork ne correspond pas à cette intégration.

Exercice 11 – Tags et push forcé

Publication d'un tag. Dans la configuration usuelle, un simple push de branche ne publie pas un nouveau tag annoté. La commande `git push origin v1.1.0` publie ce tag ; `git ls-remote -tags origin` permet de le vérifier. Le tag annoté possède son propre objet et une référence déréférencée vers son commit cible, `f0cc473`.

Q11.1 – Pourquoi force-with-lease plutôt que force ?

`-force-with-lease` vérifie que la référence distante correspond à la valeur attendue avant de la remplacer. Il refuse donc l'opération si un changement distant inattendu est détecté. `-force` n'offre pas cette protection.

On ne force pas main, ni les branches partagées ou protégées. L'amend et le push forcé sont réservés ici à une branche personnelle. La démonstration du refus après amend et de la réussite avec lease a été effectuée sur le dépôt distant local, avant publication de la branche corrigée `feature/avis`.

Vérifications

- `npm test` : 6 tests réussis.
- Livraison : Casablanca 30, Rabat 35, Marrakech 40 et Fes 40 MAD ; ville inconnue : HTTP 404.
- Promos : BIENVENUE10 à 10 %, RAMADAN20 à 20 %.
- Le PDF de l'énoncé reste non suivi ; le document de remise est fourni séparément.

Graphe final du dépôt

Commande exécutée depuis main :

```
git log --oneline --graph --all --decorate
```

Référence de capture : `fb6ea4b` sur main, avant publication de cette version du document. Les commits ultérieurs de publication ne font pas partie de cette capture.

Le fichier `graphe-git.txt` joint contient la sortie intégrale de la commande, sans retrait de commits.

```
* fb6ea4b (HEAD -> main, origin/main, origin/HEAD) docs(tp1): actualiser les references du livrable
* 153c88f docs(tp1): publier le livrable PDF et ses sources LaTeX
* 72dc70c docs: retirer le rapport du depot
* 91010c5 docs(tp1): confirmer la publication et les verifications distantes
| * d83f1b9 (origin/feature/avis, feature/avis) docs: retirer le rapport de la branche avis
| * ba48bd0 docs: ajout des avis clients
|/
* f0cc473 (tag: v1.1.0) docs(tp1): reponses et bilan des exercices Git
*   ea385df Merge upstream/main: integrer la recette QA
|\
| * c923300 (upstream/main) ReadMe Update
* | 450465b style: changer la couleur principale en bleu canard
* | 52fe0a0 docs: clarifier la conservation du panier en memoire
* |   8edf2d4 Merge fix/taux-ramadan: conserver le taux PO de 20 %
|\ \
| * | 383a058 fix(promo): taux Ramadan annonce par le marketing
* | | 0d91b02 docs(promo): preciser le taux Ramadan valide par la PO
|/ /
* |   09d2a94 Merge branch 'feature/livraison'
|\ \
| * | 080a49c feat(livraison): frais de livraison par ville
| * | 5333466 (upstream/feature/livraison) feat(livraison): frais de livraison par ville
| |/
* | a09b892 feat(promo): codes promo et poussez. BIENVENUE10 et RAMADAN20
* | 9a4c713 modif : readme
* | c442feb add livraison frais
|/
*   c0e265d (upstream/feature/promo) Fusion avec le dépôt GitHub initial
|\
| * 865adb5 Initial commit
* add0021 premier commit Artishop
```